

---

# Le tri rapide et le tri par fusion

Diviser pour régner

Tri en place • Stabilité • Complexité

CPGE deuxième année — filières MP / PSI / TSI

---

Cette section fait suite à celle étudiée en première année. L'objectif est d'explorer des algorithmes de tri plus efficaces, tel que le **tri rapide** et le **tri par fusion**, et de mettre l'accent sur la notion de **tri en place** et de **stabilité**.

|                       |                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tri rapide</b>     | Définir le principe de l'algorithme. L'implémenter. Comparer sa complexité avec celles des autres algorithmes. Discuter le choix du pivot. |
| <b>Tri par fusion</b> | Définir le principe de la méthode. L'implémenter. Comparer sa complexité avec celles des autres méthodes.                                  |

**Tri en place** — l'algorithme n'utilise qu'une mémoire auxiliaire  $\mathcal{O}(1)$  ou  $\mathcal{O}(\log n)$  : il réorganise le tableau sans en fabriquer une copie.

**Tri stable** — deux éléments de même clé restent dans leur ordre d'origine. Indispensable dès qu'on trie successivement sur plusieurs critères.

**Complexité** — on compte le nombre de *comparaisons* entre éléments, dans le meilleur cas, dans le pire cas, et en moyenne.

Rappel de première année : sélection, insertion et bulles coûtent  $\Theta(n^2)$  comparaisons dans le pire cas.

# L'idée commune : diviser pour régner

- **Diviser** : découper le tableau en deux parties ;
- **Régner** : trier chaque partie *récurivement* ;
- **Combiner** : reconstituer le tableau trié.

|                | Diviser                                  | Combiner                              |
|----------------|------------------------------------------|---------------------------------------|
| Tri par fusion | facile : couper au milieu                | difficile : fusionner ( $\Theta(n)$ ) |
| Tri rapide     | difficile : partitionner ( $\Theta(n)$ ) | rien à faire                          |

**Les deux algorithmes se partagent le travail de façon opposée.**

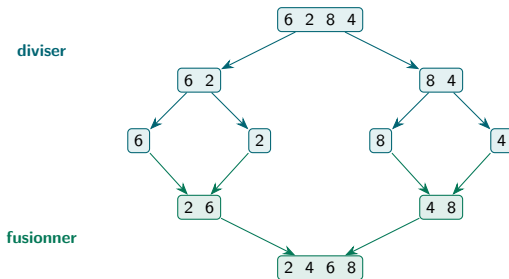
---

# Le tri par fusion

---

- 1 Le tri par fusion
- 2 Le tri rapide
- 3 Comparaison

Fusionner deux listes *déjà triées* est facile : on compare les deux premiers éléments et on prend le plus petit. D'où l'algorithme : **couper en deux, trier chaque moitié, fusionner**.



# L'étape de fusion

```
1 def fusion(u, v):
2     """u et v sont trieées ; renvoie la liste trieée de tous leurs elements."""
3     res = []
4     i, j = 0, 0
5     while i < len(u) and j < len(v):
6         if u[i] <= v[j]:
7             res.append(u[i]); i += 1
8         else:
9             res.append(v[j]); j += 1
10    return res + u[i:] + v[j:]
```

Chaque tour de boucle place définitivement un élément dans `res` : la fusion de deux listes totalisant  $n$  éléments coûte au plus  $n - 1$  comparaisons, donc  $\Theta(n)$ .

```
1 def tri_fusion(t):  
2     n = len(t)  
3     if n <= 1:                # cas de base  
4         return t  
5     m = n // 2                # diviser  
6     return fusion(tri_fusion(t[:m]), # regner  
7                   tri_fusion(t[m:])) # puis combiner
```

## Terminaison — fonction de mesure

La taille  $|t|$  est un entier positif qui décroît strictement à chaque appel ( $m < n$  et  $n - m < n$ ) : on atteint nécessairement le cas de base  $n \leq 1$ .

## Correction (récurrence sur $n$ )

Les deux moitiés sont triées par hypothèse de récurrence, et la fusion de deux listes triées renvoie une liste triée contenant exactement les mêmes éléments.

Soit  $C(n)$  le nombre de comparaisons. La division coûte 0, la fusion coûte au plus  $n$  :

$$C(n) = 2 C\left(\frac{n}{2}\right) + \Theta(n), \quad C(1) = 0.$$



$\log_2 n$  niveaux, chacun de coût  $\Theta(n) \implies C(n) = \Theta(n \log n)$ , **dans tous les cas.**

---

|                    |                                                          |
|--------------------|----------------------------------------------------------|
| Meilleur cas       | $\Theta(n \log n)$                                       |
| Cas moyen          | $\Theta(n \log n)$                                       |
| Pire cas           | $\Theta(n \log n)$ — <b>aucune entrée ne le ralentit</b> |
| Mémoire auxiliaire | $\Theta(n)$ : <b>n'est pas un tri en place</b>           |
| Stabilité          | <b>stable</b> grâce au test $u[i] \leq v[j]$             |

---

**Atout.** Une complexité garantie et la stabilité : c'est le tri de référence lorsqu'on veut une performance sûre, ou lorsque les données sont lues séquentiellement (fichiers, listes chaînées).

**Limite.** Il faut recopier les données :  $\Theta(n)$  de mémoire supplémentaire.

---

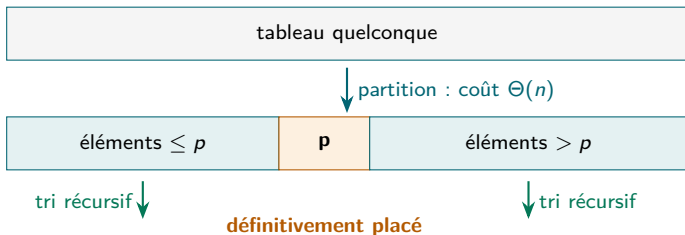
# Le tri rapide

---

- 1 Le tri par fusion
- 2 Le tri rapide**
- 3 Comparaison

# Principe

On choisit un élément, le **pivot**. On **partitionne** le tableau : à gauche les éléments plus petits que le pivot, à droite les plus grands. Le pivot est alors à *sa place définitive*, et il ne reste qu'à trier récursivement les deux côtés — **sans rien recombinaer**.



```
1 def partition(t, g, d):
2     """Partitionne t[g..d] autour du pivot t[d] ; renvoie la place du pivot."""
3     p = t[d]
4     i = g # t[g..i-1] contient les elements <= p
5     for j in range(g, d):
6         if t[j] <= p:
7             t[i], t[j] = t[j], t[i]
8             i += 1
9     t[i], t[d] = t[d], t[i] # on place enfin le pivot
10    return i
```

Un seul parcours du tableau, avec une comparaison par tour : la partition de  $n$  éléments coûte  $n - 1$  comparaisons, donc  $\Theta(n)$ , et n'utilise **aucune mémoire supplémentaire**.

```
1 def tri_rapide(t, g=0, d=None):
2     if d is None:
3         d = len(t) - 1
4     if g < d:                                # cas de base : 0 ou 1 element
5         k = partition(t, g, d)               # diviser
6         tri_rapide(t, g, k - 1)              # regner a gauche
7         tri_rapide(t, k + 1, d)              # regner a droite
8     return t
```

## Terminaison — fonction de mesure

Le pivot est *exclu* des deux appels récurifs : la taille de la tranche  $d - g + 1$  décroît donc strictement à chaque appel.

## Correction (récurrence sur la taille de la tranche)

Après la partition,  $t[k]$  est à sa place et tout élément à sa gauche lui est inférieur ou égal, tout élément à sa droite lui est supérieur. Si les deux tranches sont triées, le tout est trié.

# Complexité : tout dépend du pivot

## Le meilleur cas : le pivot coupe en deux

$C(n) = 2 C(n/2) + n$  d'où  $C(n) = \Theta(n \log n)$ , comme le tri par fusion.

## Le pire cas : le pivot est toujours le plus grand (ou le plus petit)

C'est ce qui arrive si le tableau est *déjà trié* : une tranche vide, une tranche de taille  $n - 1$ .

$$C(n) = C(n - 1) + (n - 1) \implies C(n) = \frac{n(n - 1)}{2} = \Theta(n^2).$$

## Le cas moyen

Sur une entrée aléatoire, on démontre que  $C_{\text{moy}}(n) \approx 1,39 n \log_2 n = \Theta(n \log n)$  : seulement 39 % de plus que l'optimum.

Le pire cas vient d'un pivot mal choisi. Trois parades classiques :

**1. Pivot aléatoire** — tirer au hasard l'indice du pivot puis l'échanger avec  $t[d]$ . Aucune entrée particulière n'est plus « mauvaise » qu'une autre : le  $\Theta(n \log n)$  devient vrai *en moyenne pour toute entrée*.

**2. Médiane de trois** — prendre la médiane de  $t[g]$ ,  $t[(g + d)/2]$  et  $t[d]$ . Élimine le pire cas « tableau déjà trié », de loin le plus fréquent en pratique.

**3. Bascule** — pour une tranche de moins de 15 éléments, terminer par un tri par insertion, plus rapide sur les petites tailles.

---

|                    |                                                        |
|--------------------|--------------------------------------------------------|
| Meilleur cas       | $\Theta(n \log n)$                                     |
| Cas moyen          | $\Theta(n \log n)$ , environ $1,39 n \log_2 n$         |
| Pire cas           | $\Theta(n^2)$ (tableau déjà trié, pivot extrême)       |
| Mémoire auxiliaire | $\Theta(\log n)$ (pile d'appels) : <b>tri en place</b> |
| Stabilité          | <b>non stable</b> : les échanges se font à distance    |

---

**À ne pas dire.** « Le tri rapide est en  $\Theta(n \log n)$  » est incomplet : il l'est **en moyenne**, et il est en  $\Theta(n^2)$  dans le pire cas. C'est pourtant le plus rapide en pratique, car il travaille en place, sans recopie de tableau.

---

# Comparaison

---

- 1 Le tri par fusion
- 2 Le tri rapide
- 3 Comparaison**

# Comparaison des cinq algorithmes

| Tri       | Meilleur           | Moyen              | Pire               | Mémoire          | En place | Stable |
|-----------|--------------------|--------------------|--------------------|------------------|----------|--------|
| Sélection | $\Theta(n^2)$      | $\Theta(n^2)$      | $\Theta(n^2)$      | $\Theta(1)$      | oui      | non    |
| Insertion | $\Theta(n)$        | $\Theta(n^2)$      | $\Theta(n^2)$      | $\Theta(1)$      | oui      | oui    |
| Bulles    | $\Theta(n)$        | $\Theta(n^2)$      | $\Theta(n^2)$      | $\Theta(1)$      | oui      | oui    |
| Fusion    | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n)$      | non      | oui    |
| Rapide    | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n^2)$      | $\Theta(\log n)$ | oui      | non    |

**Ordres de grandeur pour  $n = 10^6$  :** environ  $5 \cdot 10^{11}$  comparaisons pour un tri quadratique (plus d'une heure) contre  $2 \cdot 10^7$  pour un tri en  $n \log n$  (une fraction de seconde).

### Quatre points

1. Les deux algorithmes appliquent **diviser pour régner**, mais placent l'effort à des endroits opposés : la fusion pour l'un, la partition pour l'autre.
2. Le **tri par fusion** est en  $\Theta(n \log n)$  *toujours*, et il est *stable* ; en revanche il n'est pas *en place* ( $\Theta(n)$  de mémoire).
3. Le **tri rapide** est *en place* et le plus rapide en pratique, mais il est en  $\Theta(n^2)$  dans le pire cas et il n'est pas stable.
4. Le **choix du pivot** (aléatoire, médiane de trois) est ce qui rend le tri rapide fiable.

1. Dérouter à la main le tri par fusion, puis le tri rapide, sur le tableau  $[5, 3, 8, 1, 9, 2]$ . Compter les comparaisons dans chaque cas.
2. Écrire une version de `tri_rapide` avec pivot aléatoire, et vérifier expérimentalement qu'un tableau déjà trié n'est plus un cas défavorable.
3. Montrer sur un exemple que le tri rapide n'est pas stable.
4. Comparer les temps d'exécution des cinq tris pour  $n = 10^2, 10^3, 10^4$  (module `time`), et tracer les courbes obtenues.
5. Modifier le tri par fusion pour qu'il compte en plus le nombre d'inversions du tableau initial, sans changer sa complexité.

---

# Fin

Tri rapide et tri par fusion — CPGE MP / PSI / TSI

---